Amauri Hopewell

Southern New Hampshire University

CS-499-13445-M01 Computer Science Capstone

October 4, 2026

Milestone Three: Algorithms and Data Structures Enhancement

1. What is the artifact, and when was it created?

This is the ABCU course catalog from CS-300 DSA: Analysis and Design, 2025 C-3 (May–June 2025). The original is Proj2.cpp in the original/ folder of this zip and at https://github.com/AmauriHopewell/CS300. It is one C++ file. It loads a CSV of course number, title, and a list of prerequisite numbers, keeps a row only if every prerequisite exists in the file, and used to store everything in a std::vector. Search was a linear scan. Print-all walked that vector after stable_sort. In Project 1 I also wrote runtime analysis for a hash table and a binary search tree. I did not put those structures in the shipped code. This milestone is that implementation, plus the defects the Module Two review found in the same file.

2. Why this artifact, and how was it improved?

I picked this file because the original assignment was data-structure choice. The vector program works for a few hundred advising rows. It does not show that I can change the structure when search is the operation an advisor actually repeats. The Module One plan was to keep the loader and the menu, keep the vector as the CS-300 baseline, add a BST and a hash map keyed by course number, print all with an in-order walk, search with the hash map, and time all three (Hopewell, 2026). That is what is in enhanced/.

CourseBST is hand-rolled: insert, find, in-order walk, and a destructor that frees nodes. Search uses std::unordered_map, which is the STL hash table. Compare mode (menu 4) runs the same lookup 50,000 times on the vector, the tree, and the hash, and prints microseconds. Menu 5 prints the Big-O table in the program so the demo does not depend on me reciting it. Menu 6 prints a topological schedule with Kahn's algorithm so a course never appears before its prerequisites. Ties go by course number. On the sample file, CSCI301 and CSCI350 both land before CSCI400.

I also fixed the bugs that made timings dishonest. The original CSV loop never pushed the last cell unless the line ended with a comma; CSCI250 in ABCU_Advising_Program_Input.csv has no trailing comma and still loads with prerequisite CSCI101. operator== and operator> now match their names. The prereq sort uses Course& so the sort is kept. file.is_open is checked after open. The menu reads a local int, not argc. A row whose prerequisite is missing (BADPREREQ) is dropped. If I had inserted the already-sorted vector into the BST, the tree would have been a linked list. The tree is filled median-first from that sorted vector so height stays logarithmic (Cormen et al., 2009).

On this machine, 129 courses, optimized build: hash was the fastest structure. For ELEC120, which sits late in the vector, 50,000 trials took about 24,500 µs on the vector, 3,400 µs on the BST, and 450 µs on the hash. For CSCI200, near the front of the vector, the sequential scan was cheap. Three copies of the catalog cost more memory than the original vector. For this size that cost is acceptable. Hash wins single-course lookup. The tree is there for sorted listing (Hopewell, 2026).

3. Course outcomes from Module One

The Module One plan said this enhancement would show outcome 3 first, with 2 and 4 next to it. That still holds. Outcome 3 is in the running program: vector versus BST versus hash, with times and a table, plus topological order as the leftover item from Figure 2. Outcome 2 is the compare-mode output, menu 5, and this write-up. Outcome 4 is C++, chrono, and unordered_map used as tools instead of guessing. Outcome 1 is a GitHub branch after I take comments, the same pattern as CS-320. Outcome 5 is light here: is_open, std::string, no C buffers. The security work that actually closes the gap is still the CS-320 auth filter and the CS-340 credential move. I am not changing the coverage map. The 4-2 directions said to wait for instructor comments before putting this on the ePortfolio (Southern New Hampshire University [SNHU], n.d.), so Pages still links the original CS-300 repo.

4. What I learned, and what got in the way

The parser was the first hole. The original character loop reset the cell on a comma and never flushed the leftover string, so the last field vanished. Search then looked honest while a last prerequisite was gone. The sample row CSCI250 exists so that case is visible. The prereq sort was for (Course c : courses), which copies. I switched to Course&.

The BST fill would have made compare mode a lie. Inserting keys in sorted order builds a spine, so “O(log n)” on the screen would have been a linear pointer walk, and slower than the vector because the vector is contiguous. Median-first insert from the sorted array is the fix I used. Compile with /O2 as the README says; an unoptimized build makes pointer chasing in the tree look worse than it is.

Topo was marked optional in the plan. I added it after the three structures were working. Kahn’s algorithm is O(V + E). If the emitted order is shorter than the catalog, the program reports a

cycle and stops. This sample has no cycle. I did not add a GUI. The numbers on menu 4 are the product.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Hopewell, A. (2026, September 6). CS-499 Module One assignment: ePortfolio enhancement plan [Unpublished manuscript]. Southern New Hampshire University.

Southern New Hampshire University. (n.d.). CS 499 milestone three guidelines and rubric: Enhancement two algorithms and data structure.

Acknowledgment of generative AI use

I used Grok (xAI) in October 2026 to help organize this narrative against the Milestone Three prompts and to format the Word file. The program in enhanced/ is my CS-300 Proj2.cpp plus the structures and fixes listed above. I ran the load, search, compare, and topological-schedule paths on ABCU_Advising_Program_Input.csv before keeping the course counts and timings in this document.